DESENVOLVIMENTO DE APLICATIVOS MOBILE

MÓDULO 03

Kotlin Assíncrono: Coroutines e Flow

Thread principal, funções suspensas, dispatchers, concorrência estruturada, Flow e StateFlow



Prof. Dr. Francisco
IFMA – Campus São Luís – Monte Castelo
Departamento de Eletroeletrônica (DEE)

1 de novembro de 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar por que operações lentas não podem rodar na *thread* principal.
- Escrever funções `suspend` e iniciar coroutines com `launch` e `async`.
- Escolher o *dispatcher* adequado com `withContext`.
- Aplicar concorrência estruturada, cancelamento e tratamento de exceções.
- Produzir e consumir fluxos de dados com `Flow`.
- Expor estado observável com `StateFlow` e eventos com `SharedFlow`.

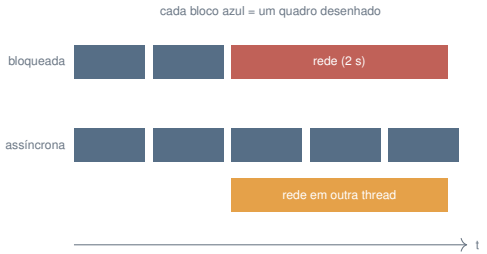
Roteiro do módulo

1. Por que programação assíncrona?
2. Fundamentos de coroutines
3. Flow: fluxos de dados
4. StateFlow e SharedFlow
5. Fechamento

1
Por que programação assíncrona?

A *thread* principal (UI thread)

- Todo app tem uma **thread principal** que desenha a tela e responde a toques.
- Para 60 Hz, cada quadro precisa ficar pronto em cerca de **16 ms**.
- Rede, banco de dados, arquivos e processamento pesado levam de dezenas de ms a vários segundos.
- Se a thread principal ficar bloqueada por $\sim$5 s, o sistema mostra o diálogo **ANR** (*Application Not Responding*).



Regra de ouro

Nunca faça I/O nem cálculo pesado na thread principal. Desde o Android 3.0, uma requisição de rede na thread principal gera `NetworkOnMainThreadException`.

Callbacks *versus* coroutines

Callbacks (*callback hell*)

```
login(usuario) { token ->
  buscarBancadas(token) { bancadas ->
    buscarLeituras(bancadas[0]) { leituras ->
      runOnUiThread {
        mostrar(leituras)
      }
    }
  }
}
// erros? cancelamento? cada nível trata o seu
```

Coroutines

```
scope.launch {
  val token = login(usuario)          // suspende
  val bancadas = buscarBancadas(token)
  val leituras = buscarLeituras(bancadas[0])
  mostrar(leituras)                   // de volta à
      UI
}
// try/catch normal, cancelamento automático
```

Coroutine

Uma computação que pode ser **suspensa** e **retomada** sem bloquear a thread. É muito mais leve que uma thread: dá para ter milhares ao mesmo tempo. O código assíncrono fica com cara de código sequencial.

2
Fundamentos de coroutines

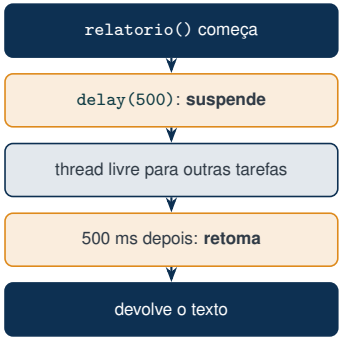
Funções `suspend`

Suspensão

```
suspend fun lerTemperatura(): Double {
  delay(500)          // suspende, NÃO bloqueia
  return 24.7
}

suspend fun relatorio(): String {
  val t = lerTemperatura()   // pode chamar suspend
  return "Temperatura: $t"
}

fun naoSuspensa() {
  // lerTemperatura()  // ERRO: só dentro de
  //                   // coroutine ou de suspend
}
```



Dica

`suspend` não cria thread nem torna nada assíncrono sozinho: só indica que a função **pode** suspender.

Construtores: launch, async e runBlocking

Iniciando coroutines

```
// launch: "dispare e esqueça", devolve Job
val job: Job = scope.launch {
  salvarLeitura(leitura)
}
job.cancel()

// async: devolve Deferred<T>, use await()
scope.launch {
  val t = async { lerTemperatura() }
  val u = async { lerUmidade() }
  // as duas rodam em paralelo
  mostrar(t.await(), u.await())
}

// runBlocking: BLOQUEIA a thread.
// Só em main() de exemplos e em testes antigos.
fun main() = runBlocking {
  println(relatorio())
}
```

Construtor	Retorno	Uso
`launch`	`Job`	tarefa sem resultado
`async`	`Deferred<T>`	resultado em paralelo
`runBlocking`	`T`	ponte com código bloqueante

Erro comum

Usar `runBlocking` no código do app trava a thread principal, que é justamente o que as coroutines evitam.

Dispatchers e `withContext`

Dispatcher	Quando usar
`Dispatchers.Main`	atualizar a UI; chamadas rápidas
`Dispatchers.IO`	rede, banco, arquivos (I/O bloqueante)
`Dispatchers.Default`	CPU: ordenar, filtrar, FFT, JSON grande

Main-safety

Uma função é *main-safe* quando pode ser chamada da thread principal com segurança, porque ela própria troca de dispatcher. Room, Retrofit e Firebase já são main-safe.

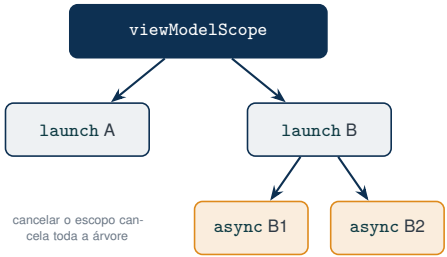
Repositório main-safe

```
class LeituraRepository(
  private val io: CoroutineDispatcher =
      Dispatchers.IO
) {
  suspend fun exportarCsv(dados: List<Leitura>):
      File =
    withContext(io) {                // troca de
      thread
      val f = File(dir, "leituras.csv")
      f.writeText(dados.joinToString("\n") {
        "${it.instante};${it.valor}"
      })
      f
    }                                // volta ao
      chamador
}
```

Dica

Receber o dispatcher no construtor permite trocá-lo por um dispatcher de teste (Módulo 11).

Concorrência estruturada



- Toda coroutine nasce dentro de um **CoroutineScope**.
- O pai espera os filhos terminarem.
- Cancelar o pai **cancela todos os filhos**.
- Uma exceção em um filho cancela o pai e os irmãos (exceto com `SupervisorJob`).

Resultado

Nenhuma coroutine fica “solta” consumindo bateria e rede depois que a tela que a criou é fechada.

Escopos no Android

Os três escopos do dia a dia

```
// 1) No ViewModel: vive enquanto o ViewModel viver
class PainelViewModel : ViewModel() {
  fun atualizar() = viewModelScope.launch {
    _estado.value = repo.carregar()
  }
}

// 2) Em Compose: ligado à composição
@Composable
fun BotaoSincronizar(onSync: suspend () -> Unit) {
  val scope = rememberCoroutineScope()
  Button(onClick = { scope.launch { onSync() } }) {
    Text("Sincronizar")
  }
}

// 3) Em Activity/Fragment: ligado ao ciclo de vida
lifecycleScope.launch { /* ... */ }
```

Escopo	Cancelado quando
`viewModelScope`	o ViewModel é destruído
`rememberCoroutineScope`	o composable sai da tela
`LaunchedEffect`	a chave muda ou sai da tela
`lifecycleScope`	a Activity é destruída

Evite `GlobalScope`

Coroutines em `GlobalScope` não são canceladas automaticamente: causam vazamentos e trabalho desnecessário.

Cancelamento e exceções

Cancelamento é cooperativo

```
val job = scope.launch(Dispatchers.Default) {
  for (amostra in amostras) {
    ensureActive()      // ou: if (!isActive) return
    processar(amostra)  // laço de CPU
  }
}
job.cancel()

// Timeout
val r = withTimeoutOrNull(3_000) {
  conectarSensor()
} ?: "sensor não respondeu"
```

Dica

Funções da biblioteca (`delay`, `withContext`, chamadas Room/Retrofit) já verificam o cancelamento.

Tratando falhas

```
viewModelScope.launch {
  try {
    val dados = repo.carregar()
    _estado.value = Estado.Sucesso(dados)
  } catch (e: CancellationException) {
    throw e            // nunca engula!
  } catch (e: IOException) {
    _estado.value = Estado.Erro("Sem conexão")
  }
}

// Filhos independentes: falha de um
// não cancela os outros
val escopo = CoroutineScope(
  SupervisorJob() + Dispatchers.Main
)
```

3
Flow: fluxos de dados

O que é um `Flow`

- Uma `suspend fun` devolve **um** valor. Um **Flow** emite **vários valores ao longo do tempo**.
- É **frio** (*cold*): o código só roda quando alguém chama `collect`, e roda de novo a cada coletor.
- Ideal para leituras de sensor, atualizações do banco e mensagens em tempo real.



Simulando um sensor

```
fun temperaturaFlow(): Flow<Double> = flow {
  var t = 24.0
  while (true) {
    t += Random.nextDouble(-0.3, 0.3)
    emit(t)           // envia um valor
    delay(1_000)      // 1 Hz
  }
}

// consumindo
scope.launch {
  temperaturaFlow()
    .map { "%.1f °C".format(it) }
    .collect { texto -> println(texto) }
}
```

Operadores úteis

Pipeline reativo

```
val alertas: Flow<String> = temperaturaFlow()
  .onEach { Log.d(TAG, "bruto: $it") }
  .map { it.arredondar(1) }
  .distinctUntilChanged()         // ignora repetidos
  .filter { it > 30.0 }
  .map { "Alerta: $it °C" }
  .catch { e -> emit("Erro: ${e.message}") }
  .flowOn(Dispatchers.Default)    // upstream em Default

// Combinar dois fluxos
val conforto = combine(tempFlow, umidFlow) { t, u ->
  indiceDeCalor(t, u)
}

// Busca com espera de digitação
consultaFlow
  .debounce(300)
  .flatMapLatest { termo -> repo.buscar(termo) }
```

Operador	Efeito
`map`, `filter`	transformar e selecionar
`onEach`	efeito colateral (log)
`distinctUntilChanged`	descarta valores repetidos
`debounce(ms)`	espera o fluxo “acalmar”
`sample(ms)`	um valor a cada período
`combine`	junta os últimos valores de vários fluxos
`flatMapLatest`	troca de fluxo e cancela o anterior
`catch`	trata exceções do upstream
`flowOn`	muda o dispatcher do upstream
`take`, `first`	limita a quantidade

Convertendo callbacks em Flow: `callbackFlow`

Sensor do aparelho como Flow

```
fun SensorManager.leituras(tipo: Int): Flow<FloatArray> =
  callbackFlow {
    val sensor = getDefaultSensor(tipo)
    val listener = object : SensorEventListener {
      override fun onSensorChanged(e: SensorEvent) {
        trySend(e.values.clone())   // envia ao Flow
      }
      override fun onAccuracyChanged(s: Sensor?, a: Int) {}
    }
    registerListener(listener, sensor,
      SensorManager.SENSOR_DELAY_UI)
    // obrigatório: limpa ao cancelar a coleta
    awaitClose { unregisterListener(listener) }
  }
```

Quando usar

APIs baseadas em *listeners*: sensores, localização, BLE, Firebase, MQTT. O `callbackFlow` transforma tudo isso em `Flow`, com cancelamento correto.

Atenção

Esquecer o `awaitClose` deixa o listener registrado, gastando bateria mesmo depois que ninguém mais observa. Voltaremos a esse padrão nos Módulos 09 e 10.

4
StateFlow e SharedFlow

StateFlow: estado observável

Padrão do ViewModel

```
class PainelViewModel(
  private val repo: LeituraRepository
) : ViewModel() {
  // mutável e privado / somente leitura e público
  private val _ultima = MutableStateFlow<Double?>(null)
  val ultima: StateFlow<Double?> = _ultima.asStateFlow()

  fun simular() {
    _ultima.value = 25.3     // emite para os coletores
    _ultima.update { (it ?: 0.0) + 0.1 }  // atômico
  }

  // Converter um Flow frio em StateFlow
  val media: StateFlow<Double> = repo.leituras()
    .map { lista -> lista.map { it.valor }.average() }
    .stateIn(
      scope = viewModelScope,
      started = SharingStarted.WhileSubscribed(5_000),
      initialValue = 0.0
    )
}
```

StateFlow

Fluxo **quente** que sempre tem um valor atual (`.value`). Novos coletores recebem imediatamente o último valor. Valores iguais em sequência não são reemitidos.

`WhileSubscribed(5_000)`

Mantém o fluxo ativo por 5 s após o último coletor sair. Assim uma rotação de tela não reinicia a consulta, mas sair da tela para tudo.

Na UI (Módulo 05)

```
val v by vm.ultima
  .collectAsStateWithLifecycle()
```

`SharedFlow` e comparação

Eventos de uso único

```
private val _eventos = MutableSharedFlow<String>()
val eventos = _eventos.asSharedFlow()

fun salvar() = viewModelScope.launch {
  repo.salvar()
  _eventos.emit("Leitura salva!")   // ex.: Snackbar
}
```

Dica

A recomendação atual é modelar até mensagens como parte do estado (ex.: `mensagem: String?` no *UiState*) e limpá-las depois de exibidas. Use `SharedFlow` quando o evento realmente não for estado.

	Flow	**StateFlow**	**SharedFlow**
Tipo	frio	quente	quente
Valor atual	não	sim (`.value`)	não (ou *replay*)
Valor inicial	—	obrigatório	—
Coletores	cada um roda o produtor	compartilham	compartilham
Uso típico	dados do repositório	estado da tela	eventos

5
Fechamento

Resumo

Resumo

- A thread principal só desenha a UI; I/O e CPU pesada vão para `Dispatchers.IO`/`Default`.
- `suspend` marca funções que podem suspender; `launch` e `async` criam coroutines.
- Use os escopos com ciclo de vida (`viewModelScope`, `rememberCoroutineScope`) e nunca `GlobalScope`.
- Cancelamento é cooperativo; relance `CancellationException`.
- `Flow` representa vários valores no tempo; operadores montam pipelines reativos.
- `StateFlow` expõe o estado da tela; `stateIn` converte fluxos frios.

Exercícios

Exercício 1 — Paralelismo

Escreva três funções `suspend` que simulam leituras com `delay` de 1 s, 2 s e 3 s. Meça o tempo total chamando-as em sequência e depois com `async`. Explique a diferença.

Exercício 2 — Flow

Crie um `Flow<Int>` que simula um ADC de 12 bits a 10 Hz. Monte um pipeline que converte para volts, calcula a média de janelas de 10 amostras e emite alerta acima de 3,0 V.

Exercício 3 — Cancelamento

Inicie um laço de CPU com `launch(Dispatchers.Default)` sem `ensureActive()` e cancele após 100 ms. O que acontece? Corrija.

LabMonitor — Etapa 3

Implemente `SimuladorSensor` com um `Flow<Leitura>` por bancada e um `PainelViewModel` que expõe a última leitura e a média dos últimos 60 s via `StateFlow`.

Para aprofundar

- Coroutines no Android: `https://developer.android.com/kotlin/coroutines`
- Boas práticas com coroutines: `https://developer.android.com/kotlin/coroutines/coroutines-best-practices`
- Guia oficial do Kotlin: `https://kotlinlang.org/docs/coroutines-guide.html`
- Flow no Android: `https://developer.android.com/kotlin/flow`
- StateFlow e SharedFlow: `https://developer.android.com/kotlin/flow/stateflow-and-sharedflow`

Fim do Módulo 03

Próximo: Módulo 04 — Interfaces com Jetpack Compose: composables, layouts, modifiers, listas e Material 3.